Machine Learning Intelligent Chip Design

Homework1 Implementation of AlexNet in SystemC

Description

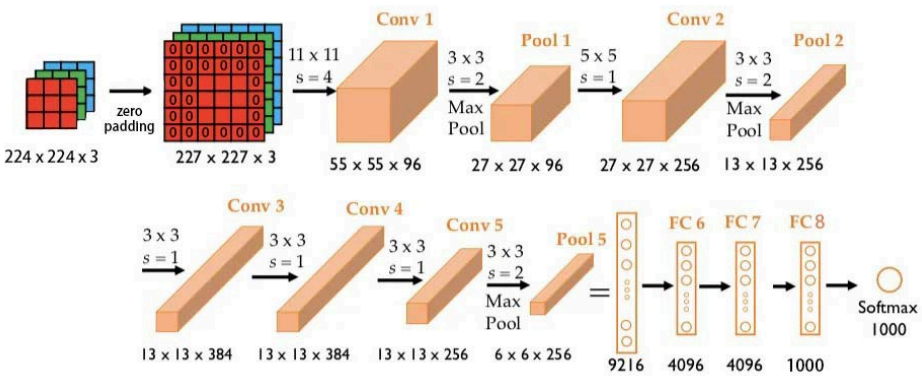
Implement the AlexNet CNN architecture using SystemC. The model consists of convolutional layers, max-pooling layers, and fully connected layers, following its original design for image classification tasks.

Implementation Details

- **Network Architecture**:
  You are provided with the network architecture details, including the number of layers, the size of input and output feature maps, filter sizes, and strides.
- **Input Data**:
  Input data for the network will be provided to you. Each input image will have the appropriate dimensions compatible with the network's input layer.
- **Weights and Biases**:
  The weights and biases for each layer of the network will be provided. These parameters are essential for the convolutional and fully connected layers' computations.
- **Simulation**:
  Once you have implemented the network in SystemC, you should simulate the network to obtain the predicted output. This process involves passing the input data through the network layers, applying convolutional operations, activation functions, and pooling, followed by fully connected layers, until the final output is obtained.

Alexnet Training Model

The AlexNet model is shown in the diagram below. Before the input image enters the first convolutional layer, **zero padding** should be applied: **two rows on the top and left, and one row on the bottom and right**. Additionally, since the results need to be displayed as probabilities, please apply softmax at the final stage.



The Pre-trained AlexNet Model Information

Layer	Type	Description	Output Dimension
0	Input Layer	Zero Padding 224 to 227	227x227x3
1	Convolutional Layer 1 + ReLU	64 kernels of 11x11, stride 4	55x55x64
1b	Max Pooling 1	3x3, stride 2	27x27x64
2	Convolutional Layer 2 + ReLU	192 kernels of 5x5, stride 1, padding 2	27x27x192
2b	Max Pooling 2	3x3, stride 2	13x13x192
3	Convolutional Layer 3 + ReLU	384 kernels of 3x3, stride 1, padding 1	13x13x384
4	Convolutional Layer 4 + ReLU	256 kernels of 3x3, stride 1, padding 1	13x13x256
5	Convolutional Layer 5 + ReLU	256 kernels of 3x3, stride 1, padding 1	13x13x256
5b	Max Pooling 5	3x3, stride 2	6x6x256
6	Fully Connected Layer 6 + ReLU	4096 neurons	4096
7	Fully Connected Layer 7 + ReLU	4096 neurons	4096
8	Fully Connected Layer 8	1000 neurons	1000
9	Softmax Layer	Converts logits to probabilities	1000

Provided Data Description

Values in the pre-train model in Pytorch are floating points with 16 digits after the decimal. We export these values as txt file for you. Values in these txt files are floating point but rounded to the sixth decimal place.

- **Model layer parameters**

  conv1_bias.txt
  conv1_weight.txt
  conv2_bias.txt
  conv2_weight.txt
  conv3_bias.txt
  conv3_weight.txt
  conv4_bias.txt
  conv4_weight.txt
  conv5_bias.txt
  conv5_weight.txt
  fc6_bias.txt
  fc6_weight.txt
  fc7_bias.txt
  fc7_weight.txt
  fc8_bias.txt
  fc8_weight.txt

- **imagenet_classes.txt**
  https://gist.github.com/ageitgey/4e1342c10a71981d0b491e1b8227328b
- **Input Data**

dog.txt

cat.txt

Input image and weight reading format

In this assignment, you need to read the image and weight txt files provided by the TA to perform calculations. The reading method follows the **raster scan order**, as illustrated in the diagram below.

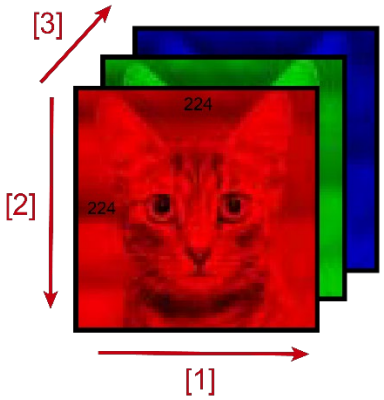
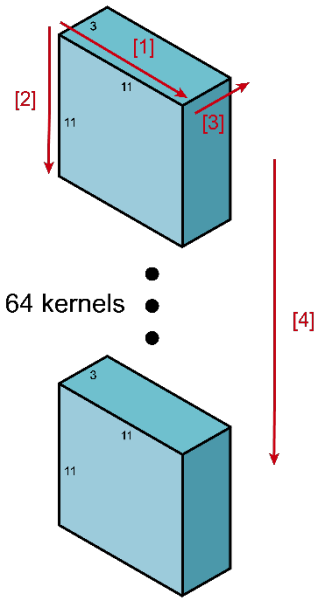


Image reading format



Weight reading format

Output Result Format

- **Simulation results of AlexNet executed in Python**
  This part is for reference only; you do not need to run Python yourself.
  Dog

```
In [20]: runfile('C:/Users/micha/Desktop/use_dataset_alexnet_model_instance.py', wdir='C:/Users/micha/Desktop')
Predicted class: golden retriever
```

  Cat

```
In [9]: runfile('C:/Users/micha/Desktop/use_dataset_alexnet_model_instance.py', wdir='C:/Users/micha/Desktop')
Predicted class: Egyptian cat
```

- **Simulation results of AlexNet executed in SystemC**
  Dog:

```
Top 5 classes:
==================================================
  idx |       val | possibility | class name
--------------------------------------------------
  207 |   16.5945 |     38.6274 | golden retriever
  175 |   15.5697 |     13.8611 | otterhound
  220 |   15.3619 |     11.2604 | Sussex spaniel
  163 |   15.0027 |     7.86243 | bloodhound
  219 |   14.5932 |     5.22078 | cocker spaniel
==================================================
```

Cat:

```
Top 5 classes:
==================================================
  idx |       val | possibility | class name
--------------------------------------------------
  285 |   20.2067 |      96.3815 | Egyptian cat
  281 |   16.1368 |      1.64619 | tabby
  282 |   15.7339 |      1.10019 | tiger cat
  287 |   14.7909 |      0.42848 | lynx
  728 |   14.4119 |      0.29332 | plastic bag
==================================================
```

The example output above only displays the top 5 labels with the highest probability. However, in this assignment, you need to display the **top 100 labels with the highest probability**, and the **output format and layout must be identical to the example provided by the TA**.For example, both **val and possibility should be 6-digit numbers**.

Below, we provide formatting code for reference. Please note that this code is for reference only. If you choose to use it, make sure to modify variable names and other details as needed.

```
cout << "Top 5 classes:" << endl;
cout << "==================================================" << endl;
cout << right << setw(5) << "idx"
     << " | " << setw(8) << "val"
     << " | " << setw(11) << "possibility"
     << " | " << "class name" << endl;
cout << "--------------------------------------------------" << endl;

for (int i = 0; i < 5; i++) {
    file.clear(); // Clear any potential error flags
    file.seekg(0, ios::beg); // Seek back to the beginning of the file
    int index = top_5_val[i].second;
    string line;
    for (int j = 0; j <= index; j++) {
        getline(file, line);
    }
    cout << right << setw(5) << index
         << " | " << setw(8) << top_5_val[i].first
         << " | " << setw(11) << (top_5_pos[i].first) // Assuming softmax outputs probabilities
         << " | " << line << endl;

}
cout << "==================================================" << endl;
```

Implement Notes

The purpose of this assignment is just to make students familiar with SystemC, so as long as the execution results are correct, we will not restrict how students implement it.

Here are some tips for your reference:

- You can use one `SC_MODULE` to implement the entire module, or implement each layer with different `SC_MODULE`.
- You can use `sc_signal` to connect different modules.
- We strongly recommend building a monitor module to receive output and print out the execution results.

Submission Guidelines & Grading Policy

- **The grading breakdown for this assignment is as follows:**
  - **Report : 30%**
    - Simulation results demonstrate the predicted output for the provided input data.
    - Your implementation approach, challenges faced, and any observations or insights gained during the implementation and simulation process.
  - **Simulation Result : 70%**
    - Only the two test datasets provided by the TA will be evaluated, **each worth 35 points**. The simulation results must **display the correct output on the terminal, and the formatting must match the provided example exactly.**
- **For the code submission, please use the compression and submission command provided by the TA. The command will be announced later.**
- **Please submit the report file to the new E3. The name of your report file is report_mlchipXXX.pdf (XXX is your account ID). If the file violates the naming rule and the file format, 10 points will be deducted.**
- **Ensure that your code is well-commented and organized for clarity and understanding.**
- **Plagiarism is forbidden, otherwise you will get 0 point!!!**